Laboratorio 3 Grupo 5: bitácora.

Se presenta los prompts utilizados para el armado del código para ‘Multiplicaciones sucesivas POW’ a través de Assembly de la LC-3.

El orden de los prompts con su respectivo código, generado por la IA (se utilizó Gemini primeramente y los últimos arreglos fueron a través de Claude), está en orden de pedido. Cada código fue evaluado para depurar sus errores y mejorar diferentes objetivos, que se especifican en el prompt, a través de https://wchargin.com/lc3web/ .

El código final y sus especificaciones se encentran a partir de la página 6.

Cada ejecución de cada uno de los códigos se presenta a través de capturas de pantalla acompañado de una breve explicación del error depurado y mejora en el código para que realice de la mejor manera lo esperado.

1. Prompt: hola, podes generar un código para la LC-3 codificado en assembly para resolver multiplicaciones sucesivas teniendo las siguientes pautas: todo en 16 bits, el overflow donde el rango máximo se define a través del punto Q que creas conveniente para realizar de la manera más eficiente la tarea que te di y las etiquetas que no salgan fuera del rango de las instrucciones de la LC-3, la multiplicación debe de hacerse a través de la suma sucesiva, el valor a multiplicarse por sí mismo u otro valor lo ingresa el usuario por teclado, para valores tanto enteros como decimales, para los números decimales vas a tener que aplicar el formato Q, hace el código guiándote SI O SI de estos archivos PDF para que el ensamblador lo admita y ya no tenga más errores.

   Código:

```
; =================================================================
; Programa: Multiplicaciones Sucesivas en LC-3 con Formato Q (Q4)
; Arquitectura: 16 bits
; Basado en guías oficiales de LC3Tools y rutinas TRAP de I/O
; =================================================================

.ORIG x3000

; --- 1. Inicialización de Constantes y Máscaras ---
    LD R6, ASCII      ; R6 = x30 (máscara para convertir a ASCII)
    LD R5, NEGASCII   ; R5 = xFFD0 (-x30, máscara para convertir ASCII a número)

; --- 2. Ingreso del Primer Número (Base A en formato Q4) ---
    LEA R0, PROMPT1
    TRAP x22          ; PUTS: Mostrar mensaje de solicitud de A
    TRAP x23          ; IN: Leer carácter del teclado (almacenado en R0)
    TRAP x21          ; OUT: Eco del carácter en pantalla
    ADD R1, R0, x0    ; Mover carácter ingresado a R1
    ADD R1, R1, R5    ; Convertir de ASCII a valor numérico (A en R1)
```

```
; --- 3. Ingreso del Segundo Número (Multiplicador B / Veces) ---
    LEA R0, PROMPT2
    TRAP x22        ; PUTS: Mostrar mensaje de solicitud de B
    TRAP x23        ; IN: Leer segundo carácter del teclado
    TRAP x21        ; OUT: Eco del carácter en pantalla
    ADD R2, R0, x0    ; Mover carácter a R2
    ADD R2, R2, R5    ; Convertir de ASCII a valor numérico (B en R2)

; --- 4. Multiplicación por Sumas Sucesivas ---
    AND R3, R3, x0    ; R3 = 0 (Acumulador del producto)
    ADD R2, R2, x0    ; Comprobar si el multiplicador B es cero
    BRz FIN_MULT      ; Si B == 0, salta directo al resultado

LOOP_MULT
    ADD R3, R3, R1    ; Acumulador = Acumulador + A (Suma sucesiva)
    ADD R2, R2, #-1   ; Decrementar el contador de iteraciones B
    BRp LOOP_MULT     ; Si B > 0, repetir el ciclo de suma

FIN_MULT
    ; --- 5. Despliegue del Resultado ---
    LEA R0, MESG_RES
    TRAP x22        ; PUTS: Mostrar etiqueta de resultado

    ; Conversión del resultado numérico acumulado en R3 a formato ASCII
    ADD R0, R3, R6    ; Sumar x30 para obtener el carácter ASCII correspondiente
    TRAP x21        ; OUT: Imprimir el resultado en la consola

    HALT            ; Fin de ejecución

; =============================================================================
; Declaración de Datos y Constantes (Ubicados cerca para cumplir
; estrictamente con el rango de saltos relativos y LEA de la LC-3)
; =============================================================================
ASCII       .FILL x30     ; Máscara ASCII (+x30)
NEGASCII    .FILL xFFD0   ; Máscara negativa ASCII (-x30)
PROMPT1     .STRINGZ "\nIngrese valor A (Q4): "
PROMPT2     .STRINGZ "\nIngrese multiplicador B: "
MESG_RES    .STRINGZ "\nResultado (Q4): "
```

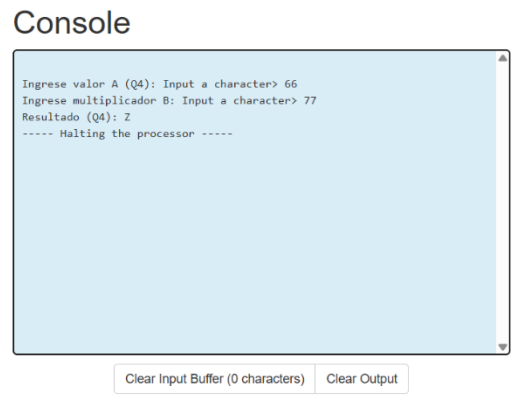


Ilustración 1. Código 1 y ejecución

```
.END
```

Ejecución y errores:

Se ingresó los valores A=6 y B=7, en la consola se muestran duplicados y el resultado se muestra en ASCII, el resultado es erróneo.

2. Prompt: edita el código para que cuando haya un error lo muestre en la interfaz, además que los resultados sea en decimal no de otra manera y los ingresos también, todo el procesamiento si o si debe de ser por detrás de lo que se muestra, porque cuando el usuario ingresa los dos valores siendo un 2 y un 5, muestra dos veces el valor ingresado, siendo 22 y un 55,

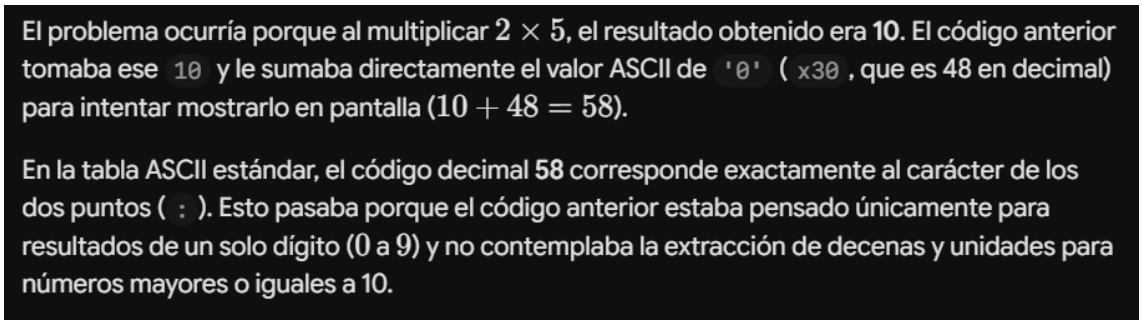


Ilustración 2. Errores

además el resultado de eso muestra dos puntos ':', en tabla ASCII tiene un valor numérico de lista 58, no sé qué está haciendo el código .

Explicación del error:

Código corregido:

```
.ORIG x3000
LD R6, ASCII
LD R5, NEGASCII
LEA R0, PROMPT1
TRAP x22
TRAP x23
TRAP x21
ADD R1, R0, x0
ADD R1, R1, R5
LEA R0, PROMPT2
TRAP x22
TRAP x23
TRAP x21
ADD R2, R0, x0
ADD R2, R2, R5
AND R3, R3, x0
ADD R2, R2, x0
BRz FIN_MULT
LOOP_MULT
ADD R3, R3, R1
ADD R2, R2, #-1
BRp LOOP_MULT
FIN_MULT
LEA R0, MESG_RES
TRAP x22
AND R4, R4, x0
LOOP_TENS
LD R7, NEGTEN
ADD R3, R3, R7
BRn END_TENS
```

```
ADD R4, R4, #1
BR LOOP_TENS
END_TENS
LD R7, TEN
ADD R3, R3, R7
ADD R4, R4, x0
BRz SKIP_TENS
ADD R0, R4, R6
TRAP x21
SKIP_TENS
ADD R0, R3, R6
TRAP x21
HALT
ASCII .FILL x30
NEGASCII .FILL xFFD0
TEN .FILL #10
NEGTEN .FILL #-10
PROMPT1 .STRINGZ "\nIngrese valor A (Q4): "
PROMPT2 .STRINGZ "\nIngrese multiplicador B: "
MESG_RES .STRINGZ "\nResultado (Q4): "
.END
```

Luego de ensamblar el código, se solucionó el problema para admitir el resultado de 2 cifras, pero el usuario al ingresar los valores a multiplicarse, se duplicaban.

3. Solución ingreso duplicado, lógica de procesamiento interno en background y la salida admite hasta 5 cifras para el resultado.

```
.ORIG x3000
LD R6, ASCII
LD R5, NEGASCII
LEA R0, PROMPT1
TRAP x22
TRAP x23
ADD R1, R0, x0
ADD R1, R1, R5
LEA R0, PROMPT2
TRAP x22
TRAP x23
ADD R2, R0, x0
ADD R2, R2, R5
AND R3, R3, x0
ADD R2, R2, x0
BRz FIN_MULT
LOOP_MULT
ADD R3, R3, R1
ADD R2, R2, #-1
BRp LOOP_MULT
FIN_MULT
LEA R0, MESG_RES
TRAP x22
AND R4, R4, x0
LOOP_TENS
LD R7, NEGTEN
```

```
ADD R3, R3, R7
BRn END_TENS
ADD R4, R4, #1
BR LOOP_TENS
END_TENS
LD R7, TEN
ADD R3, R3, R7
ADD R4, R4, x0
BRz SKIP_TENS
ADD R0, R4, R6
TRAP x21
SKIP_TENS
ADD R0, R3, R6
TRAP x21
HALT
ASCII .FILL x30
NEGASCII .FILL xFFD0
TEN .FILL #10
NEGTEN .FILL #-10
PROMPT1 .STRINGZ "\nIngrese valor A (Q4): "
PROMPT2 .STRINGZ "\nIngrese multiplicador B: "
MESG_RES .STRINGZ "\nResultado (Q4): "
.END
```

Consola

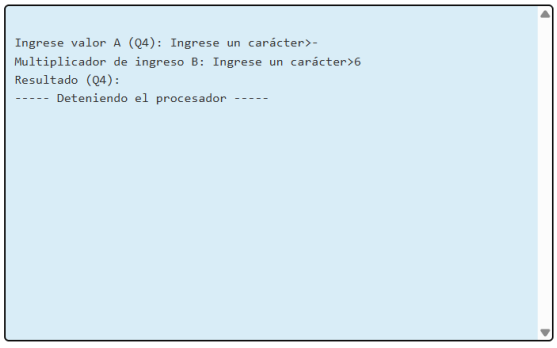


Ilustración 3. Error ingreso

4. Problema: el usuario ingresaba dos valores, sin duplicado, pero el ensamblador no admite valor decimales ni mayores a 10, toma el signo negativo como valor de la tabla ASCII, pero el resultado queda incompleto en la muestra.

Al tener muchos problemas como Gemini para la programación, se recurrió a Claude IA de la siguiente forma: primero se le dio las instrucciones para generar un código en c++ lo más básico posible, sin librerías, que realice la multiplicación de un numero A a través de la suma sucesiva de B veces (el valor B indica cuantas veces se debe de sumar A para que se multiplique A por B).

Código en c++ por Claude:

```cpp
#include <iostream>
using namespace std;

float multiplicar(float A, float B) {
    float r = 0;
    if (B < 0) {
        A = -A;
        B = -B;
    }
    int veces = B;
    for (int i = 0; i < veces; i++) {
        r = r + A;
    }
    return r + A * (B - veces);
}

int main() {
    float A, B;
    cout << "Ingrese A: ";
    cin >> A;
    cout << "Ingrese B: ";
    cin >> B;
    cout << "Resultado: " << multiplicar(A, B) << endl;
    return 0;
}
```

Este código en c++ se utilizó para verificar el funcionamiento de la multiplicación por suma sucesiva,

luego se le pidió a Claude que, en base a este código, genere en assembly el funcionamiento tal cual.
Para ello, se le indico ciertas pautas en un prompt.

Prompt: para assembly tenés que contemplar los siguientes casos: números decimales, negativos, la LC-3 funciona ESTRICTAMENTE con 16 bits, no debe de ocurrir ningún desbordamiento y para los números con coma en bits debes de ocupar el formato Q más conveniente, eficiente y "liviano".

Código final:

```
.ORIG x3000
    LD R6, PILA
    LEA R0, MSGA
    PUTS
    JSR LEER
    ST R1, AQ
    LEA R0, MSGB
    PUTS
    JSR LEER
    ADD R2, R1, #0      ; R2 = B
    LD R1, AQ           ; R1 = A
    JSR MULT            ; R0 = A*B
    ST R0, RQ
    LEA R0, MSGR
    PUTS
    LD R3, RQ
    BRzp MOSTRAR
    LD R0, ASC_MENOS
    OUT
    LD R3, RQ
    NOT R3, R3
    ADD R3, R3, #1
MOSTRAR AND R4, R4, #0
    LD R5, NEG256
SEP   ADD R2, R3, R5
    BRn FIN_SEP
    ADD R3, R2, #0
    ADD R4, R4, #1
    BRnzp SEP
FIN_SEP ST R3, FRAC8
    ADD R0, R4, #0
    AND R5, R5, #0
    ADD R5, R5, #1
    JSR IMPRIMIR
    LD R0, ASC_PUNTO
    OUT
    JSR DECIMALES
    LD R0, ASC_LF
    OUT
    HALT
```

```
MSGA    .STRINGZ "Ingrese A: "
MSGB    .STRINGZ "Ingrese B: "
MSGR    .STRINGZ "Resultado: "

LEER    ST R7, GUARDA7
        AND R0, R0, #0
        ST R0, SIGNO
        ST R0, NDIG
        ADD R0, R0, #-1
        ST R0, FCNT
        AND R1, R1, #0
        AND R2, R2, #0
        GETC
        OUT
        LD R3, NEG_MENOS
        ADD R3, R0, R3
        BRnp CICLO
        ADD R3, R3, #1
        ST R3, SIGNO
        GETC
        OUT
CICLO   LD R3, NEG_ENTER
        ADD R3, R0, R3
        BRz FIN_LEER
        LD R3, NEG_PUNTO
        ADD R3, R0, R3
        BRnp DIGITO
        LD R3, FCNT
        BRzp ERR_ENT
        AND R3, R3, #0
        ST R3, FCNT
        BRnzp SIGUE
DIGITO  LD R3, NEG_CERO
        ADD R0, R0, R3
        BRn ERR_ENT
        ADD R3, R0, #-9
        BRp ERR_ENT
        LD R3, NDIG
        ADD R3, R3, #1
        ST R3, NDIG
        LD R3, FCNT
        BRzp FRAC
        ADD R3, R1, #-12
        BRp ERR_DESB
        ADD R3, R1, R1
        ADD R4, R3, R3
        ADD R4, R4, R4
        ADD R1, R4, R3
        ADD R1, R1, R0
        LD R3, NEG127
        ADD R3, R1, R3
```

```
        BRp ERR_DESB
        BRnzp SIGUE
FRAC    ADD R3, R3, #1
        ST R3, FCNT
        ADD R3, R3, #-3
        BRp ERR_ENT
        ADD R3, R2, R2
        ADD R4, R3, R3
        ADD R4, R4, R4
        ADD R2, R4, R3
        ADD R2, R2, R0
SIGUE   GETC
        OUT
        BRnzp CICLO
FIN_LEER LD R3, NDIG
        BRz ERR_ENT
        LD R3, FCNT
        BRzp PAD_N
        AND R3, R3, #0
PAD_N   NOT R3, R3
        ADD R3, R3, #1
        ADD R3, R3, #3
PAD     ADD R3, R3, #0
        BRz FIN_PAD
        ADD R4, R2, R2
        ADD R5, R4, R4
        ADD R5, R5, R5
        ADD R2, R5, R4
        ADD R3, R3, #-1
        BRnzp PAD
FIN_PAD AND R3, R3, #0
        AND R4, R4, #0
        ADD R4, R4, #8
        LD R5, NEG1000
BITS    ADD R3, R3, R3
        ADD R2, R2, R2
        ADD R0, R2, R5
        BRn NOBIT
        ADD R2, R0, #0
        ADD R3, R3, #1
NOBIT   ADD R4, R4, #-1
        BRp BITS
        LD R5, NEG500
        ADD R0, R2, R5
        BRn SIN_RED
        ADD R3, R3, #1
SIN_RED AND R4, R4, #0
        ADD R4, R4, #8
D256    ADD R1, R1, R1
        ADD R4, R4, #-1
        BRp D256
        ADD R1, R1, R3
```

```
      BRn ERR_DESB
      LD R0, SIGNO
      BRz LISTO
      NOT R1, R1
      ADD R1, R1, #1
LISTO   LD R7, GUARDA7
      RET

PILA    .FILL x4000
NEG_MENOS .FILL xFFD3
NEG_PUNTO .FILL xFFD2
NEG_ENTER .FILL xFFF6
NEG_CERO .FILL xFFD0
NEG127  .FILL #-127
NEG256  .FILL #-256
NEG500  .FILL #-500
NEG1000 .FILL #-1000
ASC_MENOS .FILL x2D
ASC_PUNTO .FILL x2E
ASC_CERO .FILL x30
ASC_LF  .FILL x0A
GUARDA7 .BLKW #1
SIGNO   .BLKW #1
NDIG    .BLKW #1
FCNT    .BLKW #1
FRAC8   .BLKW #1
AQ      .BLKW #1
RQ      .BLKW #1

ERR_ENT LD R0, ASC_LF
      OUT
      LEA R0, MSGE1
      PUTS
      HALT
ERR_DESB LD R0, ASC_LF
      OUT
      LEA R0, MSGE2
      PUTS
      HALT
MSGE1   .STRINGZ "Error: entrada invalida"
MSGE2   .STRINGZ "Error: fuera de rango Q7.8 (+-127.99)"

;
================================================================
======
; MULT - Multiplicacion Q7.8 por suma sucesiva (SUBRUTINA)
;   Entrada : R1 = A    (Q7.8 con signo, 16 bits)
;           R2 = B    (Q7.8 con signo, 16 bits)
;   Salida  : R0 = A*B  (Q7.8 con signo, 16 bits)
;   Preserva: R1..R5 (los guarda en la pila) y R6. No toca R7 (no hay JSR).
;   Desborde: salta a ERR_DESB (el resultado no entra en Q7.8)
;
```

```
;  Metodo (todo con sumas, sin multiplicar):
;    |B| = BI + BF/256   (BI = parte entera, BF = parte fraccionaria)
;    |A|*|B| = |A| sumado BI veces          -> fase 1
;            + (|A|/256) sumado BF veces     -> fase 2
;    con |A|/256 = AI + AF/256: cada suma de la fase 2 agrega AI al
;    resultado y AF a un acumulador que "acarrea" 1 al llegar a 256.
;    Asi nunca se necesita un valor mayor a 16 bits.
;
================================================================================
MULT   ADD R6, R6, #-5       ; reservar 5 palabras en la pila
       STR R1, R6, #0
       STR R2, R6, #1
       STR R3, R6, #2
       STR R4, R6, #3
       STR R5, R6, #4
       ; ---- signos y valores absolutos ----
       AND R5, R5, #0        ; R5 = cantidad de operandos negativos
       ADD R1, R1, #0
       BRzp A_POS
       NOT R1, R1
       ADD R1, R1, #1        ; R1 = |A|
       BRn ERR_DESB          ; x8000 no tiene opuesto en 16 bits
       ADD R5, R5, #1
A_POS  ADD R2, R2, #0
       BRzp B_POS
       NOT R2, R2
       ADD R2, R2, #1        ; R2 = |B|
       BRn ERR_DESB
       ADD R5, R5, #1
B_POS  AND R0, R0, #0
       ADD R5, R5, #-1       ; con 1 solo negativo R5 queda en 0
       BRnp SGN_SET
       ADD R0, R0, #1        ; R0 = 1 -> resultado negativo
SGN_SET ST R0, SGN
       ; ---- separar |B| = BI*256 + BF (restas sucesivas de 256) ----
       ADD R3, R2, #0        ; R3 = resto (termina siendo BF)
       AND R4, R4, #0        ; R4 = BI
       LD R5, NEG256
DIV_B  ADD R2, R3, R5
       BRn FIN_B
       ADD R3, R2, #0
       ADD R4, R4, #1
       BRnzp DIV_B
FIN_B  ST R3, BF             ; BF (0..255)
       ; ---- fase 1: sumar |A|, BI veces ----
       AND R0, R0, #0        ; resultado = 0
       ADD R4, R4, #0
       BRz DIV_A
FASE1  ADD R0, R0, R1        ; resultado += |A|
       BRn ERR_DESB
       ADD R4, R4, #-1
```

```
    BRp FASE1
    ; ---- separar |A| = AI*256 + AF ----
DIV_A  ADD R3, R1, #0       ; R3 = resto (termina siendo AF)
    AND R4, R4, #0       ; R4 = AI
    LD R5, NEG256
DA_LOOP ADD R2, R3, R5
    BRn FIN_A
    ADD R3, R2, #0
    ADD R4, R4, #1
    BRnzp DA_LOOP
    ; ---- fase 2: sumar |A|/256, BF veces ----
FIN_A  AND R5, R5, #0       ; R5 = acumulador de fracciones
    LD R1, BF           ; R1 = contador (|A| ya no hace falta)
    BRz FIN_M
FASE2  ADD R0, R0, R4       ; resultado += AI
    BRn ERR_DESB
    ADD R5, R5, R3       ; acumulador += AF
    LD R2, NEG256
    ADD R2, R5, R2       ; acumulador - 256
    BRn SIG
    ADD R5, R2, #0       ; acumulador -= 256
    ADD R0, R0, #1       ; acarreo: resultado += 1
    BRn ERR_DESB
SIG   ADD R1, R1, #-1
    BRp FASE2
    ; ---- signo y retorno ----
FIN_M  LD R1, SGN
    BRz VUELVE
    NOT R0, R0
    ADD R0, R0, #1       ; resultado negativo
VUELVE  LDR R1, R6, #0       ; restaurar R1..R5
    LDR R2, R6, #1
    LDR R3, R6, #2
    LDR R4, R6, #3
    LDR R5, R6, #4
    ADD R6, R6, #5
    RET
SGN   .BLKW #1           ; variables locales de MULT
BF    .BLKW #1

DECIMALES ST R7, GUARDA7
    LD R3, FRAC8
    AND R4, R4, #0
    ADD R4, R4, #4
FLOOP  ADD R1, R3, R3
    ADD R2, R1, R1
    ADD R2, R2, R2
    ADD R3, R2, R1
    AND R0, R0, #0
    LD R5, NEG256
FD    ADD R2, R3, R5
    BRn FIN_FD
```

```
        ADD R3, R2, #0
        ADD R0, R0, #1
        BRnzp FD
FIN_FD  LD R1, ASC_CERO
        ADD R0, R0, R1
        OUT
        ADD R4, R4, #-1
        BRp FLOOP
        LD R7, GUARDA7
        RET

IMPRIMIR ST R7, GUARDA7
        ADD R1, R0, #0
        AND R4, R4, #0
DIV     AND R2, R2, #0
RESTA   ADD R3, R1, #-10
        BRn FIN_DIV
        ADD R1, R3, #0
        ADD R2, R2, #1
        BRnzp RESTA
FIN_DIV LD R0, ASC_CERO
        ADD R0, R0, R1
        ADD R6, R6, #-1
        STR R0, R6, #0
        ADD R4, R4, #1
        ADD R5, R5, #-1
        ADD R1, R2, #0
        BRp DIV
        ADD R5, R5, #0
        BRp DIV
SACAR   LDR R0, R6, #0
        ADD R6, R6, #1
        OUT
        ADD R4, R4, #-1
        BRp SACAR
        LD R7, GUARDA7
        RET
        .END
```

Especificaciones: 16 bits, Q 7.8, cuanto mayor es A menor debe de ser B (y viceversa) para que no produzca un overflow, no es posible un reingreso si el usuario se equivoca, B puede ser decimal y/o negativo, no admite valores imaginarios, la multiplicación por suma sucesiva es a través de sumar A por si mismo B veces. Registros utilizados: R0, R1 y R2.

Ilustración 4. Disposición de registros

```
JSR LEER
ADD R2, R1, #0      ; R2 = B
LD  R1, AQ          ; R1 = A
JSR MULT            ; R0 = A*B
ST  R0, RQ
```